

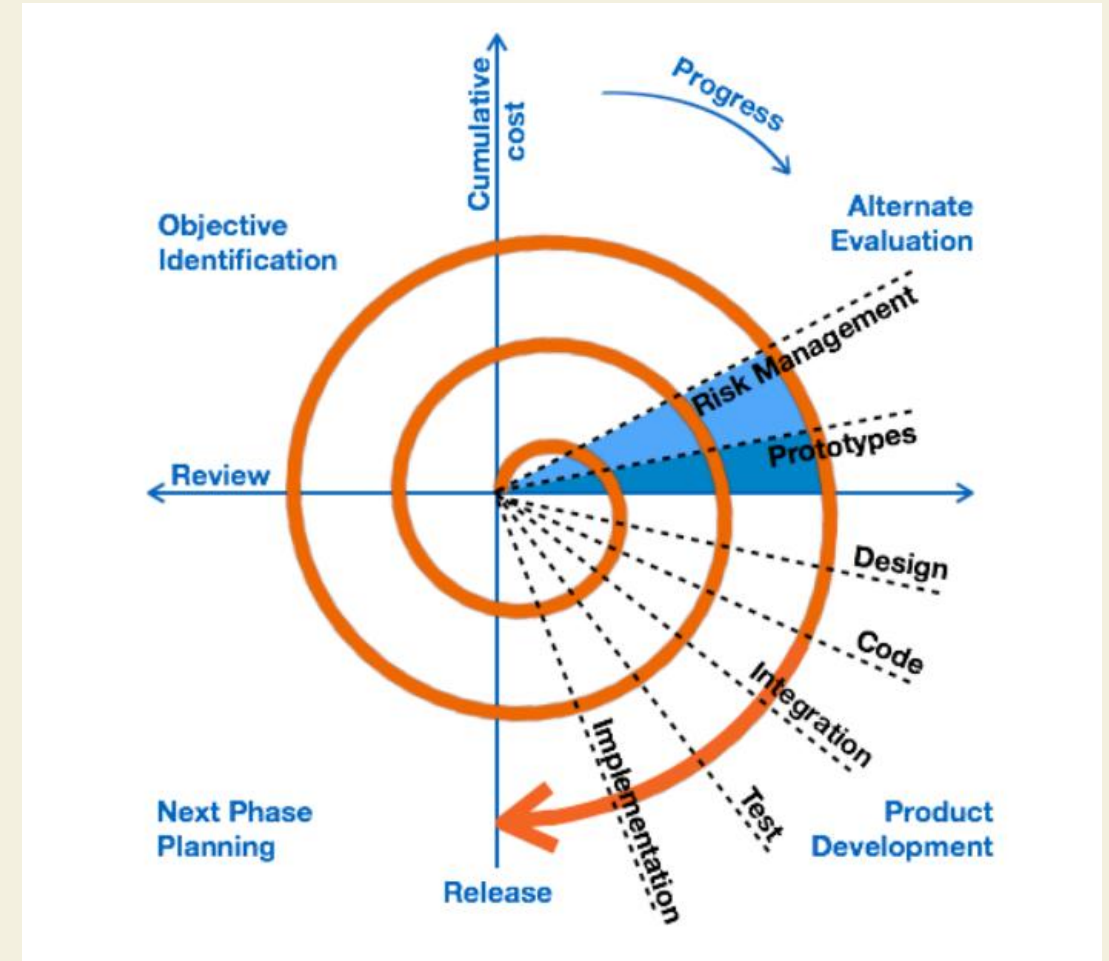
The Spiral Model

A Risk-Driven Approach in Software
Engineering

What is the Spiral Model?

It's a Software Development Life Cycle (SDLC) model that combines features of the **waterfall** and **iterative** models.

It provides a systematic, iterative approach with a strong emphasis on **risk handling**.



Origin and Key Concept

Proposed by Barry Boehm

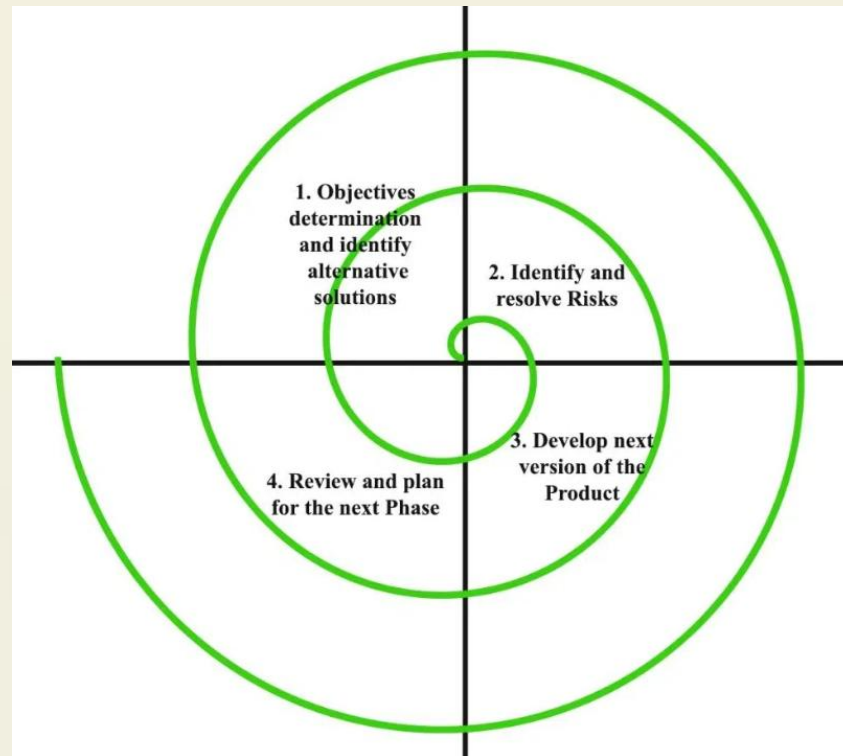
The model was first introduced to address the shortcomings of traditional models when dealing with large, complex projects.

A Risk-Driven Spiral

It's visually represented as a spiral, where each loop represents a complete software development cycle.

The Four Phases

Each spiral loop is divided into four key phases, representing the steps of the process.



Phase 1: Planning

This phase focuses on:

- Clarifying project aims
- Gathering requirements
- Identifying objectives
- Proposing alternative solutions



Phase 2: Risk Analysis

This is the most critical quadrant:

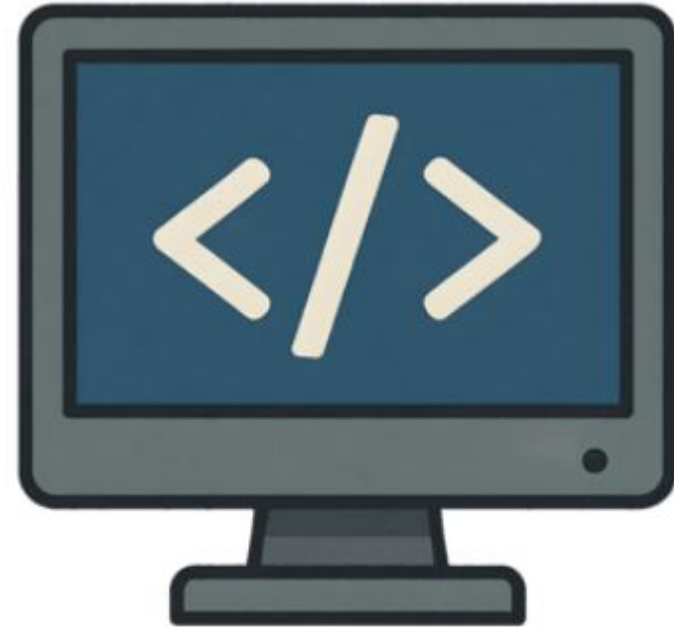
- Evaluating proposed solutions
- Identifying associated risks
- Resolving those risks
- Often involves building a **Prototype** to find the best possible solution.



Phase 3: Development

With risks addressed, development begins:

- Developing and verifying the identified features.
- This phase includes coding, integration, and testing.
- The result is the next version of the software.



Phase 4: Review

The loop concludes with evaluation:

- Customers evaluate the developed software version.
- Feedback is gathered.
- Planning for the next iteration (the next loop of the spiral) begins.



A "Meta-Model"

" It encompasses other SDLC models... and utilizes the Prototyping Model for its risk-handling approach. "

- The Spiral Model is often called a "Meta-Model" because it adapts and includes other methodologies.

Meta- Model

- The spiral model incorporates the stepwise approach of the Classical Waterfall Model.
- The spiral model uses the approach of the Prototyping Model building a prototype at the start of each phase as a risk-handling technique.
- Also, the spiral model can be considered as supporting the Evolutionary Model – the iterations along the spiral can be considered as evolutionary levels through which the complete system is built.

An Example: *Developing an E-Commerce Website*

1. First Spiral – Planning and Requirements:

In this first phase, the team gathers the basic requirements for the e-commerce website, like product listings, shopping carts, and payment options. They also identify potential risks, like security issues or scalability concerns.

To get started, they build a simple prototype, like a homepage with a basic product catalog, to see how users interact with it and spot any initial design problems.

An Example: *Developing an E-Commerce Website*

2. Second Spiral – Risk Analysis and Refining the Design

After getting feedback from the first prototype, the team moves to the next phase. They add more features and address the problems that were found earlier.

This includes verifying secure payment processing and testing how the site handles more users. They also add a basic shopping cart and user registration system, and they run tests with dummy transactions to make sure everything is secure.

An Example: *Developing an E-Commerce Website*

3. Third Spiral – Detailed Implementation

With more feedback in hand, the team adds advanced features, like order tracking, customer reviews, and a search function. They also revisit any remaining risks, especially around scalability making sure the website can handle a growing number of users.

During this phase, the team tests the site to verify it can handle large orders, especially during busy times like sales or holidays.

An Example: *Developing an E-Commerce Website*

4. Final Spiral – Full Deployment

In the last phase, the website is fully implemented, tested properly, and then launched to the public. Any remaining risks, like potential crashes or user feedback issues, are closely monitored and dealt with.

This example shows how the Spiral Model allows for continuous improvement, with feedback and risk assessment at each step to make sure the final product is solid and reliable.

Pros....

1. **Good for large projects.**
2. **Flexibility in Requirements**
3. **Customer Satisfaction**
4. **Iterative and Incremental Approach**
5. **Emphasis on Risk Management**
6. **Improved Communication**
7. **Improved Quality**



Cons....

1. **Complex**
2. **Expensive**
3. **Too much dependency on risk management**
4. **Difficulty in time management**
5. **Resource Intensive**



When to Use the Spiral Model

-  **Complex Requirements:** When requirements are complicated, ambiguous, or expected to change.
-  **High Risk:** For large, complex projects with significant uncertainty or high levels of risk. Pioneering projects.
-  **Frequent Releases:** When frequent releases or prototypes are necessary to get user feedback.
-  **Large-Scale Projects:** Best suited for mission-critical or large-scale software development.

Questions?

References

- Slides adapted from Didarul Islam Didar, Lecturer, Dept. of CSE, UIU
- [What is Spiral Model in Software Engineering? - GeeksforGeeks](#)

Thank You ^_^ !!!